

Entregable A - Orquestador Concurrente de Modelos

Simulacion asincrona de un backend que consulta multiples fuentes de IA. Incluye `asyncio.gather`, `asyncio.timeout` y `asyncio.Semaphore`. No se llaman APIs reales; la latencia se simula con `asyncio.sleep`.

1.Codigo fuente

```
"""Entregable A: Orquestador Concurrente de Modelos.
```

```
Simula la logica de un backend que consulta multiples fuentes de IA de forma eficiente, sin llamar APIs reales (solo simula latencia con asyncio.sleep).
```

```
Demo 1: tres "modelos" corren en paralelo con asyncio.gather, la ejecucion total esta acotada por asyncio.timeout(2.0).
```

```
Demo 2: 10 llamadas simuladas se lanzan a la vez pero un asyncio.Semaphore(2) limita a 2 ejecuciones concurrentes.
```

```
"""
```

```
import asyncio
import logging
import random
import time
```

```
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s.%(msecs)03d | %(message)s",
    datefmt="%H:%M:%S",
)
logger = logging.getLogger("orquestador")
```

```
INICIO_PROGRAMA = time.perf_counter()
```

```
def transcurrido() -> str:
    """Segundos transcurridos desde el arranque del programa, para logs legibles."""
    return f"t+{time.perf_counter() - INICIO_PROGRAMA:0.3f}s"
```

```
# -----
# Demo 1: gather + timeout sobre tres "modelos"
# -----
```

```
async def gpt_4_call(prompt: str) -> str:
    """Simula una llamada al modelo GPT-4 (latencia rapida)."""
    logger.info("[%s] gpt_4_call: inicio", transcurrido())
    await asyncio.sleep(1.0)
    logger.info("[%s] gpt_4_call: fin", transcurrido())
    return f"Respuesta de GPT-4 para: '{prompt}'"
```

```
async def claude_3_call(prompt: str) -> str:
    """Simula una llamada al modelo Claude 3 (latencia media)."""
    logger.info("[%s] claude_3_call: inicio", transcurrido())
    await asyncio.sleep(1.5)
    logger.info("[%s] claude_3_call: fin", transcurrido())
    return f"Respuesta de Claude 3 para: '{prompt}'"
```

```
async def local_llama_call(prompt: str) -> str:
    """Simula una llamada a un Llama local (latencia lenta, dispara el timeout)."""
    logger.info("[%s] local_llama_call: inicio", transcurrido())
    await asyncio.sleep(3.0)
    logger.info("[%s] local_llama_call: fin", transcurrido())
    return f"Respuesta de Llama local para: '{prompt}'"
```

```

async def orquestar_tres_modelos(prompt: str) -> None:
    """Dispara los tres modelos en simultaneo, acotado por un timeout global de 2s."""
    logger.info("--- Demo 1: gather + timeout(2.0) ---")
    try:
        async with asyncio.timeout(2.0):
            resultados = await asyncio.gather(
                gpt_4_call(prompt),
                claude_3_call(prompt),
                local_llama_call(prompt),
            )
            for resultado in resultados:
                logger.info("Resultado recibido: %s", resultado)
    except TimeoutError:
        logger.error(
            "[%s] Se agoto el tiempo de espera (2.0s): al menos un modelo tardo demasiado",
            transcurrido(),
        )
    logger.info("El programa sigue corriendo despues del timeout.\n")

# -----
# Demo 2: Semaphore limitando concurrencia sobre 10 llamadas
# -----

async def llamada_simulada(indice: int, semaforo: asyncio.Semaphore) -> str:
    """Simula una llamada generica a un modelo, respetando el limite del semaforo."""
    async with semaforo:
        logger.info("[%s] llamada #%d: adquiere el semaforo, inicio", transcurrido(), indice)
        duracion = random.uniform(0.3, 0.6)
        await asyncio.sleep(duracion)
        logger.info("[%s] llamada #%d: libera el semaforo, fin", transcurrido(), indice)
        return f"Resultado de la llamada #{indice}"

async def orquestar_con_semaforo(cantidad_llamadas: int = 10, limite_concurrente: int = 2) -> None:
    """Lanza N llamadas simuladas a la vez, pero solo `limite_concurrente` corren en paralelo."""
    logger.info("--- Demo 2: Semaphore(%d) sobre %d llamadas ---", limite_concurrente, cantidad_llamadas)
    semaforo = asyncio.Semaphore(limite_concurrente)
    tareas = [llamada_simulada(i, semaforo) for i in range(1, cantidad_llamadas + 1)]
    resultados = await asyncio.gather(*tareas)
    logger.info("Total de resultados recibidos: %d", len(resultados))

async def main() -> None:
    await orquestar_tres_modelos("Que es la entropia?")
    await orquestar_con_semaforo()

if __name__ == "__main__":
    asyncio.run(main())

```

2. Evidencia de ejecucion (logs por consola)

Se observa: (a) las tres corrutinas de la Demo 1 inician con timestamps casi identicos; (b) local_llama_call (3.0s) supera el timeout global de 2.0s y se captura TimeoutError sin detener el programa; (c) en la Demo 2, el Semaphore(2) permite que solo 2 de las 10 llamadas esten activas a la vez (se ve por los pares de 'inicio' seguidos de 'fin' antes de que arranque la siguiente pareja).

```
12:01:08.884 | --- Demo 1: gather + timeout(2.0) ---
12:01:08.884 | [t+0.002s] gpt_4_call: inicio
12:01:08.885 | [t+0.003s] claude_3_call: inicio
12:01:08.885 | [t+0.003s] local_llama_call: inicio
12:01:09.898 | [t+1.016s] gpt_4_call: fin
12:01:10.389 | [t+1.508s] claude_3_call: fin
12:01:10.898 | [t+2.016s] Se agoto el tiempo de espera (2.0s): al menos un modelo tardo demasiado
12:01:10.898 | El programa sigue corriendo despues del timeout.
```

```
12:01:10.898 | --- Demo 2: Semaphore(2) sobre 10 llamadas ---
12:01:10.898 | [t+2.016s] llamada #1: adquiere el semaforo, inicio
12:01:10.898 | [t+2.017s] llamada #2: adquiere el semaforo, inicio
12:01:11.271 | [t+2.389s] llamada #2: libera el semaforo, fin
12:01:11.271 | [t+2.390s] llamada #3: adquiere el semaforo, inicio
12:01:11.474 | [t+2.593s] llamada #1: libera el semaforo, fin
12:01:11.475 | [t+2.593s] llamada #4: adquiere el semaforo, inicio
12:01:11.752 | [t+2.870s] llamada #3: libera el semaforo, fin
12:01:11.752 | [t+2.870s] llamada #5: adquiere el semaforo, inicio
12:01:11.968 | [t+3.086s] llamada #4: libera el semaforo, fin
12:01:11.968 | [t+3.086s] llamada #6: adquiere el semaforo, inicio
12:01:12.231 | [t+3.349s] llamada #5: libera el semaforo, fin
12:01:12.232 | [t+3.350s] llamada #7: adquiere el semaforo, inicio
12:01:12.465 | [t+3.584s] llamada #6: libera el semaforo, fin
12:01:12.466 | [t+3.584s] llamada #8: adquiere el semaforo, inicio
12:01:12.759 | [t+3.877s] llamada #7: libera el semaforo, fin
12:01:12.759 | [t+3.878s] llamada #9: adquiere el semaforo, inicio
12:01:12.821 | [t+3.939s] llamada #8: libera el semaforo, fin
12:01:12.822 | [t+3.940s] llamada #10: adquiere el semaforo, inicio
12:01:13.114 | [t+4.232s] llamada #9: libera el semaforo, fin
12:01:13.175 | [t+4.293s] llamada #10: libera el semaforo, fin
12:01:13.175 | Total de resultados recibidos: 10
```